Migrating from psycopg2 to psycopg3: Complete Guide with Async Support for DatabaseConnection

Table of Contents {#table-of-contents}

- Migrating from psycopg2 to psycopg3: Complete Guide with Async Support for DatabaseConnection
  - Table of Contents
  - Introduction
  - Understanding Key Differences
  - Setting Up Prerequisites
  - Step 1: Creating the PSYCOPG3 Dictionary Structure
  - Step 2: Adding Asynchronous Extensions
  - Step 3: Implementing Connection Pooling Support
  - Step 4: Extending the Dictionary with DatabaseConnector Support
  - Step 5: Adding Transaction Management
  - Step 6: Integrating Async Support into DatabaseConnection Class
  - Step 7: Migrating SQL Syntax for PostgreSQL
  - Step 8: Example Implementation
    - Synchronous Database Operations
    - Asynchronous Database Operations
    - Connection Pool Example
  - Conclusion
  - Complete PSYCOPG3 Dictionary Reference

Introduction {#-introduction}

This guide provides a comprehensive approach to upgrading your database interaction layer from psycopg2 to psycopg3, with special emphasis on leveraging asynchronous capabilities. The migration will be applied to an existing `DatabaseConnection` class, enhancing it with async support through psycopg3's modern features.

psycopg3 represents a significant evolution from psycopg2, offering native async support, improved performance, and better integration with modern Python features. By upgrading to psycopg3, your applications will benefit from:

- Native async/await support for non-blocking database operations
- Enhanced connection pooling capabilities
- Improved performance through optimized data transfer
- Better error handling and diagnostics
- Support for modern Python features like type annotations

This guide will walk you through the entire migration process with practical examples and detailed explanations, providing everything you need to modernize your PostgreSQL database interactions.

Back to Table of Contents

🔄 Understanding Key Differences {#-understanding-key-differences}

Before diving into the migration process, it's important to understand the key differences between psycopg2 and psycopg3:

1. **Package Name Changes**:
   - From `psycopg2` to simply `psycopg` (there is no "psycopg3" package name)
2. **Connection Approach**:
   - psycopg2: Traditional synchronous connections only
   - psycopg3: Supports both synchronous and asynchronous connections through different classes
3. **Parameter Placeholders**:
   - Both use `%s` for parameter placeholders (unlike some other database drivers)
4. **Connection String Format**:
   - Minor differences in formatting (e.g., semicolons vs. spaces between parameters)
   - psycopg3 uses "dbname" instead of "database" in connection strings
5. **Modular Architecture**:
   - psycopg3 has a more modular structure with separate components:
     - `psycopg` - Core functionality
     - `psycopg_pool` - Connection pooling
6. **Asynchronous Support**:
   - psycopg3 provides built-in async support:
     - `psycopg.AsyncConnection` - Asynchronous connection class
     - `psycopg.AsyncCursor` - Asynchronous cursor class
     - `psycopg_pool.AsyncConnectionPool` - Asynchronous connection pool
7. **Type Annotations**:
   - Better support for Python type hints, making code more maintainable
8. **Error Handling**:
   - Improved exception hierarchy for more precise error handling
9. **Context Managers**:
   - Enhanced support for both `with` and `async with` statements for cleaner resource management
   - Transaction management through context managers
10. **Performance**:
    - Improved data transfer performance
    - Better handling of large result sets

Back to Table of Contents

Setting Up Prerequisites {#-setting-up-prerequisites}

Before starting the migration, ensure you have the following prerequisites:

1. **Python 3.7+**: For full async support, Python 3.8+ is recommended
2. **Install psycopg3**:

```
# Install psycopg3 with uv for faster installation
uv pip install "psycopg[binary]"

# Add pool support for connection pooling
uv pip install "psycopg[binary,pool]"

# If you prefer using traditional pip
pip install "psycopg[binary,pool]"
```

1. **PostgreSQL Server**: Version 10+ is recommended

2. **Basic Understanding**: Familiarity with async/await patterns in Python

Your project folder structure should be organized as follows:

```
project/
├── core/
│   └── connector.py         # Location of the DatabaseConnector class
├── templates/
│   └── constraints.py       # Contains enums like TargetDatabaseEngine
├── db_constants/
│   └── database_connection_constants.py  # Location for the PSYCOPG3 dictionary
└── .env                     # For database credentials
```

Once these prerequisites are in place, you're ready to begin the migration process.

Back to Table of Contents

Step 1: Creating the PSYCOPG3 Dictionary Structure {#-step-1-creating-the-psycopg3-dictionary-structure}

Start by creating the base dictionary structure that defines the core configuration for psycopg3. This will serve as the foundation for all database operations.

Add the following dictionary to your `database_connection_constants.py` file:

```
PSYCOPG3: Dict[str, Dict[str, Any]] = {
    "PSYCOPG3": {
        "DEFAULT_PARAMETERS": {
```

```
            "DATABASE_NAME": "NorthwindOrderSubsystem",
            "SERVER": "localhost",
            "PORT_NUMBER": 5432,  # Default PostgreSQL port
            "USERNAME": "postgres",
            "PASSWORD": "Password",
            "FORMAT_CONNECTION_STRING": {
                "LOAD_DEFAULT_PARAMETER_KEYS":
["DATABASE_NAME","SERVER","PORT_NUMBER"],
                "METHOD_LOAD_DEFAULT_PARAMETER_KEYS_DICT":
load_default_parameter_keys_dict,
                "IS_DSN_NAME_SUPPORTED": "N",
                "BASE_CONNECTION_STRING": "host={0} port={1} dbname={2} user={3}
password={4}",
                "BASE_ODBC_DSN_CONNECTION_STRING": None
            }
        },
        "FINALIZED_CONNECTION_PARAMS": {
            "LAMBDA_FINALIZED_BASE_CONNECTION_STRING": lambda d: d["PSYCOPG3"]
["DEFAULT_PARAMETERS"]["FORMAT_CONNECTION_STRING"]
["BASE_CONNECTION_STRING"].format(
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["SERVER"],
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["PORT_NUMBER"],
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["DATABASE_NAME"],
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["USERNAME"],
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["PASSWORD"]
            ),
            "LAMBDA_FINALIZED_DSN_CONNECTION_STRING": lambda d: d["PSYCOPG3"]
["DEFAULT_PARAMETERS"]["FORMAT_CONNECTION_STRING"]
["BASE_ODBC_DSN_CONNECTION_STRING"].format(
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["DSN_NAME"]
            ) if d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["FORMAT_CONNECTION_STRING"]
["IS_DSN_NAME_SUPPORTED"] == "Y" else None
        },
        "CRUDBASIC": {
            "IMPORT": "psycopg",
            "PARAMETER_PLACEHOLDER": "%s",
            "IS_USERNAME_AND_PASSWORD_REQUIRED": True
        },
        "CREATE_DATABASE_CONNECTION": {
            "CONNECTION_NOTES": "Use psycopg.connect with the appropriate
connection string for synchronous operations.",
            "LAMBDA_FINALIZED_PARAMETER_CONNECTION_STRING": lambda
finalized_parameter_connection_string: lambda:
get_db_target_engine_module('psycopg').connect(finalized_parameter_connection_stri
ng)
        }
    }
}
```

Key Components Explained:

1. **DEFAULT_PARAMETERS**: Contains the core connection parameters:

- Database credentials (server, port, username, password)
- Connection string formatting rules
- DSN support flags

2. **FINALIZED_CONNECTION_PARAMS**: Provides lambda functions for generating:
   - Formatted connection strings from the default parameters
   - DSN connection strings (if supported)
3. **CRUDBASIC**: Defines the core CRUD operation settings:
   - Import module name (`psycopg` instead of `psycopg2`)
   - Parameter placeholder format (`%s`)
   - Authentication requirements
4. **CREATE_DATABASE_CONNECTION**: Contains functions and notes for creating database connections:
   - Connection function that uses the appropriate engine module
   - Usage notes for connection creation

This base structure provides the foundation for synchronous database operations with psycopg3. The key differences from the psycopg2 configuration are the import module name and the connection string format.

Back to Table of Contents

⚡ Step 2: Adding Asynchronous Extensions {#-step-2-adding-asynchronous-extensions}

Now, extend the dictionary to include asynchronous operation support. This is one of the key advantages of psycopg3 over psycopg2.

Add the following "ASYNC_EXTENSIONS" section to the PSYCOPG3 dictionary:

```
"ASYNC_EXTENSIONS": {
    "IMPORT": "psycopg.AsyncConnection",
    "IS_SUPPORTED": "Y",
    "CONNECTION_NOTES": "Use psycopg.AsyncConnection.connect with the appropriate
connection string for asynchronous operations.",
    "LAMBDA_FINALIZED_PARAMETER_ASYNC_CONNECTION_STRING": lambda
finalized_parameter_connection_string: lambda:
get_db_target_engine_module('psycopg.AsyncConnection').connect(finalized_parameter
_connection_string),
    "ASYNC_CURSOR": {
        "IMPORT": "psycopg.AsyncCursor",
        "USAGE_NOTES": "Use 'async with conn.cursor() as cur:' pattern for cursor
operations."
    },
    "ASYNC_EXECUTE": {
        "EXECUTE_PATTERN": "await cursor.execute(query, params)",
        "FETCH_PATTERN": {
            "FETCHONE": "await cursor.fetchone()",
            "FETCHMANY": "await cursor.fetchmany(size)",
```

```
            "FETCHALL": "await cursor.fetchall()"
        }
    },
    "ASYNC_TRANSACTION": {
        "BEGIN_PATTERN": "async with conn.transaction():",
        "USAGE_NOTES": "Transactions are managed using context managers in
psycopg3."
    }
}
```

Asynchronous Extensions Explained:

1. **Basic Configuration**:
    - Import path for `AsyncConnection` from psycopg3
    - Support flag indicating that async is available
    - Usage notes for creating async connections
2. **Async Connection Function**:
    - Lambda function that creates an async connection
    - Uses the asynchronous connection class from psycopg3
3. **Async Cursor Operations**:
    - Import path for `AsyncCursor`
    - Usage pattern using async context managers
4. **Async Execution Patterns**:
    - Execution patterns for queries with parameters
    - Fetch patterns for retrieving results asynchronously
5. **Async Transaction Management**:
    - Transaction context management pattern
    - Usage notes for transaction handling

These extensions define all the necessary components for asynchronous database operations using psycopg3. The async capabilities allow for non-blocking database interactions, which can significantly improve application performance in I/O-bound scenarios.

Back to Table of Contents

🔌 Step 3: Implementing Connection Pooling Support {#-step-3-implementing-connection-pooling-support}

Connection pooling is a critical feature for production applications, allowing for efficient reuse of database connections. Add connection pooling support to the ASYNC_EXTENSIONS section:

```
"CONNECTION_POOL": {
    "IMPORT": "psycopg_pool.AsyncConnectionPool",
    "USAGE_NOTES": "Use AsyncConnectionPool for connection pooling with async
operations.",
    "LAMBDA_CREATE_POOL": lambda finalized_parameter_connection_string,
min_size=1, max_size=10: lambda:
get_db_target_engine_module('psycopg_pool').AsyncConnectionPool(finalized_paramete
r_connection_string, min_size=min_size, max_size=max_size)
}
```

Connection Pooling Implementation Explained:

1. **Import Information**:
    - Module path for the async connection pool (`psycopg_pool.AsyncConnectionPool`)
2. **Usage Notes**:
    - Guidelines for using the connection pool in asynchronous operations
3. **Pool Creation Function**:
    - Lambda function that creates an async connection pool
    - Configurable min and max pool sizes
    - Returns a function that creates the pool when called

Connection pooling is particularly valuable in web applications and microservices where many short-lived database operations occur. The pool maintains a set of reusable connections, eliminating the overhead of repeatedly establishing and closing connections.

Key benefits include:

- Reduced connection establishment overhead
- Better resource utilization
- Improved application performance
- Configurable pool sizes to match workload requirements

Back to Table of Contents

🧩 Step 4: Extending the Dictionary with DatabaseConnector Support {#-step-4-extending-the-dictionary-with-databaseconnector-support}

Now, enhance the dictionary with components to support the DatabaseConnector class that can leverage both synchronous and asynchronous operations:

```
"DATABASE_CONNECTOR_EXTENSIONS": {
    "IMPORT_DEPENDENCIES": [
        "asyncio",
        "pandas as pd",
        "prettytable",
```

```
        "typing import Dict, Any, Optional, Union, List, Tuple"
    ],
    "INITIALIZATION": {
        "CONSTRUCTOR_PARAMS": {
            "use_async": {
                "TYPE": "bool",
                "DEFAULT": "False",
                "DESCRIPTION": "Flag to determine if async mode should be used"
            }
        },
        "TRANSACTION_MANAGER_INIT": "self.transaction_manager =
AsyncTransactionManager(self) if use_async else TransactionManager(self)"
    },
    "SYNC_METHODS": {
        "connect_to_database": {
            "CODE": """
            engine_module = get_db_target_engine_module(self._get_module_name())
            connection_func = self._get_connection_func()
            self.connection = connection_func()
            return self
            """,
            "DESCRIPTION": "Create synchronous database connection"
        },
        "_get_module_name": {
            "CODE": """
            if self.target_database_engine == TargetDatabaseEngine.PSYCOPG3:
                return 'psycopg'
            # Add support for other engines
            raise ValueError(f"Unsupported database engine:
{self.target_database_engine}")
            """,
            "DESCRIPTION": "Get the module name for the selected database engine"
        },
        "_get_connection_func": {
            "CODE": """
            if self.target_database_engine == TargetDatabaseEngine.PSYCOPG3:
                conn_string = self._get_connection_string()
                return PSYCOPG3["PSYCOPG3"]["CREATE_DATABASE_CONNECTION"]
["LAMBDA_FINALIZED_PARAMETER_CONNECTION_STRING"](conn_string)
            # Add support for other engines
            raise ValueError(f"Unsupported database engine:
{self.target_database_engine}")
            """,
            "DESCRIPTION": "Get the connection function for the selected database
engine"
        },
        "execute": {
            "CODE": """
            with self.connection.cursor() as cursor:
                cursor.execute(query, parameters)
                self.rowcount = cursor.rowcount
            return self
            """,
            "DESCRIPTION": "Execute a SQL query with parameters"
```

```
        },
        "execute_query_to_dataframe": {
            "CODE": """
            with self.connection.cursor() as cursor:
                cursor.execute(query, parameters)
                columns = [desc[0] for desc in cursor.description]
                result = cursor.fetchall()
                self.dataframe = pd.DataFrame(result, columns=columns)
            return self
            """,
            "DESCRIPTION": "Execute a SQL query and store results in a dataframe"
        },
        "execute_query_to_prettytable": {
            "CODE": """
            with self.connection.cursor() as cursor:
                cursor.execute(query, parameters)
                columns = [desc[0] for desc in cursor.description]
                result = cursor.fetchall()

                pt = prettytable.PrettyTable()
                pt.field_names = columns
                for row in result:
                    pt.add_row(row)

                print(pt)
                self.prettytable = pt
            return self
            """,
            "DESCRIPTION": "Execute a SQL query and display results in a
prettytable"
        },
        "to_dataframe": {
            "CODE": """
            return self.dataframe
            """,
            "DESCRIPTION": "Return the current dataframe"
        },
        "close": {
            "CODE": """
            if self.connection:
                self.connection.close()
            return self
            """,
            "DESCRIPTION": "Close the database connection"
        }
    },
    "ASYNC_METHODS": {
        "connect_to_database_async": {
            "CODE": """
            engine_module =
get_db_target_engine_module(self._get_async_module_name())
            connection_func = self._get_async_connection_func()
            self.connection = await connection_func()
            return self
```

```
            """,
            "DESCRIPTION": "Create asynchronous database connection"
        },
        "_get_async_module_name": {
            "CODE": """
            if self.target_database_engine == TargetDatabaseEngine.PSYCOPG3:
                return 'psycopg.AsyncConnection'
            # Add support for other engines
            raise ValueError(f"Async mode not supported for
{self.target_database_engine}")
            """,
            "DESCRIPTION": "Get the async module name for the selected database
engine"
        },
        "_get_async_connection_func": {
            "CODE": """
            if self.target_database_engine == TargetDatabaseEngine.PSYCOPG3:
                conn_string = self._get_connection_string()
                return PSYCOPG3["PSYCOPG3"]["ASYNC_EXTENSIONS"]
["LAMBDA_FINALIZED_PARAMETER_ASYNC_CONNECTION_STRING"](conn_string)
            # Add support for other engines
            raise ValueError(f"Async mode not supported for
{self.target_database_engine}")
            """,
            "DESCRIPTION": "Get the async connection function for the selected
database engine"
        },
        "execute_async": {
            "CODE": """
            async with self.connection.cursor() as cursor:
                await cursor.execute(query, parameters)
                self.rowcount = cursor.rowcount
            return self
            """,
            "DESCRIPTION": "Execute a SQL query asynchronously with parameters"
        },
        "execute_query_to_dataframe_async": {
            "CODE": """
            async with self.connection.cursor() as cursor:
                await cursor.execute(query, parameters)
                columns = [desc[0] for desc in cursor.description]
                result = await cursor.fetchall()
                self.dataframe = pd.DataFrame(result, columns=columns)
            return self
            """,
            "DESCRIPTION": "Execute a SQL query asynchronously and store results
in a dataframe"
        },
        "execute_query_to_prettytable_async": {
            "CODE": """
            async with self.connection.cursor() as cursor:
                await cursor.execute(query, parameters)
                columns = [desc[0] for desc in cursor.description]
                result = await cursor.fetchall()
```

```
                pt = prettytable.PrettyTable()
                pt.field_names = columns
                for row in result:
                    pt.add_row(row)

                print(pt)
                self.prettytable = pt
            return self
            """,
            "DESCRIPTION": "Execute a SQL query asynchronously and display results
in a prettytable"
        },
        "close_async": {
            "CODE": """
            if self.connection:
                await self.connection.close()
            return self
            """,
            "DESCRIPTION": "Close the database connection asynchronously"
        }
    }
}
```

DatabaseConnector Extensions Explained:

1. **Import Dependencies**:
    - List of required modules and packages
    - Includes async support, data processing, and visualization libraries
2. **Initialization Configuration**:
    - Constructor parameters including an async mode flag
    - Transaction manager initialization based on the async mode
3. **Synchronous Methods**:
    - Connection establishment
    - Query execution
    - Result processing
    - Resource cleanup
4. **Asynchronous Methods**:
    - Async connection establishment
    - Async query execution
    - Async result processing
    - Async resource cleanup
5. **Helper Methods**:

- Module name resolution
- Connection function resolution
- Engine-specific implementation handling

These extensions provide the building blocks for updating the DatabaseConnector class to support both synchronous and asynchronous operations with psycopg3.

Back to Table of Contents

🔄 Step 5: Adding Transaction Management {#-step-5-adding-transaction-management}

Transaction management is a critical aspect of database operations. Add transaction manager implementations for both synchronous and asynchronous operations:

```
"TRANSACTION_MANAGER_EXTENSIONS": {
    "SYNC_TRANSACTION_MANAGER": {
        "INITIALIZATION": {
            "CODE": """
            def __init__(self, db_connector):
                self.db_connector = db_connector
                self.transaction_active = False
            """
        },
        "METHODS": {
            "begin_transaction": {
                "CODE": """
                if self.transaction_active:
                    raise ValueError("Transaction already in progress")

                # Set isolation level
                isolation_level_cmd = f"SET TRANSACTION ISOLATION LEVEL
{isolation_level.value}"
                with self.db_connector.connection.cursor() as cursor:
                    cursor.execute(isolation_level_cmd)

                # Begin transaction
                with self.db_connector.connection.cursor() as cursor:
                    cursor.execute("BEGIN")

                self.transaction_active = True
                return self
                """,
                "DESCRIPTION": "Begin a new transaction with specified isolation
level"
            },
            "commit": {
                "CODE": """
                if not self.transaction_active:
                    raise ValueError("No transaction in progress")

                with self.db_connector.connection.cursor() as cursor:
```

```
                    cursor.execute("COMMIT")

                self.transaction_active = False
                return self
                """,
                "DESCRIPTION": "Commit the current transaction"
            },
            "rollback": {
                "CODE": """
                if not self.transaction_active:
                    raise ValueError("No transaction in progress")

                with self.db_connector.connection.cursor() as cursor:
                    cursor.execute("ROLLBACK")

                self.transaction_active = False
                return self
                """,
                "DESCRIPTION": "Rollback the current transaction"
            }
        }
    },
    "ASYNC_TRANSACTION_MANAGER": {
        "INITIALIZATION": {
            "CODE": """
            def __init__(self, db_connector):
                self.db_connector = db_connector
                self.transaction = None
            """
        },
        "METHODS": {
            "begin_transaction_async": {
                "CODE": """
                # Set isolation level
                isolation_level_cmd = f"SET TRANSACTION ISOLATION LEVEL
{isolation_level.value}"
                async with self.db_connector.connection.cursor() as cursor:
                    await cursor.execute(isolation_level_cmd)

                # Begin transaction context
                self.transaction = self.db_connector.connection.transaction()
                await self.transaction.start()
                return self
                """,
                "DESCRIPTION": "Begin a new transaction asynchronously with
specified isolation level"
            },
            "commit_async": {
                "CODE": """
                if self.transaction:
                    await self.transaction.commit()
                    self.transaction = None
                return self
                """,
```

```
                "DESCRIPTION": "Commit the current transaction asynchronously"
            },
            "rollback_async": {
                "CODE": """
                if self.transaction:
                    await self.transaction.rollback()
                    self.transaction = None
                return self
                """,
                "DESCRIPTION": "Rollback the current transaction asynchronously"
            }
        }
    }
}
```

Transaction Manager Extensions Explained:

1. **Synchronous Transaction Manager**:
    - Initialization with state tracking
    - Transaction start with isolation level configuration
    - Explicit commit and rollback operations
    - State validation to prevent invalid operations
2. **Asynchronous Transaction Manager**:
    - Initialization with transaction object reference
    - Async transaction start with isolation level configuration
    - Async commit and rollback operations
    - Context-manager-based transaction handling

The transaction managers provide a clean API for controlling transactions in both synchronous and asynchronous contexts, ensuring proper resource management and error handling.

Back to Table of Contents

✨ Step 6: Integrating Async Support into DatabaseConnection Class {#-step-6-integrating-async-support-into-databaseconnection-class}

Now that we have defined the PSYCOPG3 dictionary with async extensions, we need to update the DatabaseConnection class to support asynchronous operations. Here's how to enhance the existing DatabaseConnection class:

1. **Update the Constructor** to support the async mode:

```
def __init__(self, target_database_engine_type: TargetDatabaseEngine,
             target_dbconnect_option: TargetDatabaseConnectionStringOption,
             key_management: Optional[str] = None,
             use_async: bool = False):
    # Existing initialization code...
```

```
    # Add async mode flag
    self._use_async = use_async

    # Initialize appropriate transaction manager based on async mode
    if self._use_async:
        self.transaction_manager = AsyncTransactionManager(self)
    else:
        self.transaction_manager = TransactionManager(self)
```

2. **Add Asynchronous Connection Method**:

```
async def connect_to_database_async(self) -> 'DatabaseConnection':
    """
    Establishes an asynchronous connection to the database using the finalized
connection parameters.

    This method retrieves the connection string from the database engine
configuration and
    attempts to connect to the database asynchronously. If the connection is
successful,
    it returns the current instance; otherwise, it raises a ConnectionError.

    Returns:
        DatabaseConnection: The current instance after successfully establishing a
connection.

    Raises:
        ConnectionError: If the connection to the database fails.
    """
    try:
        async_connection_func = self._get_async_connection_func()
        self._connection = await async_connection_func()
        return self
    except Exception as e:
        raise ConnectionError(f"Failed to connect to database asynchronously:
{e}")
```

3. **Add Helper Methods for Async Support**:

```
def _get_async_module_name(self) -> str:
    """
    Get the module name for asynchronous operations based on the target database
engine.

    Returns:
        str: The module name for async operations.

    Raises:
        ValueError: If async mode is not supported for the selected database
```

```
engine.
    """
    if self._target_database_engine_type_name == "PSYCOPG3":
        return 'psycopg.AsyncConnection'
    # Add support for other engines as needed
    raise ValueError(f"Async mode not supported for
{self._target_database_engine_type_name}")

def _get_async_connection_func(self) -> Callable:
    """
    Get the async connection function for the target database engine.

    Returns:
        Callable: A function that creates an async connection when called.

    Raises:
        ValueError: If async mode is not supported for the selected database
engine.
    """
    if self._target_database_engine_type_name == "PSYCOPG3":
        engine_config = self._get_engine_config()
        return engine_config["ASYNC_EXTENSIONS"]
["LAMBDA_FINALIZED_PARAMETER_ASYNC_CONNECTION_STRING"](
            self._finalization_connection_parameters_string
        )
    # Add support for other engines as needed
    raise ValueError(f"Async mode not supported for
{self._target_database_engine_type_name}")
```

4. **Add Asynchronous Query Execution Methods**:

```
async def execute_async(self, sql_command: str, parameters: Optional[List] = None)
-> 'DatabaseConnection':
    """
    Executes a SQL command asynchronously.

    Args:
        sql_command (str): The SQL command to execute.
        parameters (Optional[List], optional): Parameters for the SQL command.
Defaults to None.

    Returns:
        DatabaseConnection: The current instance for method chaining.

    Raises:
        Exception: If the database connection has not been established.
    """
    if self._connection is None:
        raise Exception("Connection not established.")

    async with self._connection.cursor() as cursor:
        await cursor.execute(sql_command, parameters)
```

```
            self.rowcount = cursor.rowcount
        return self

    async def execute_sql_to_dataframe_async(self, sql_command: str, index_column: str
    = None) -> 'DatabaseConnection':
        """
        Executes a SQL command asynchronously and stores the result in a dataframe.

        Args:
            sql_command (str): The SQL command to execute.
            index_column (str, optional): The column to use as the index. Defaults to
    None.

        Returns:
            DatabaseConnection: The current instance for method chaining.

        Raises:
            Exception: If the database connection has not been established.
        """
        if self._connection is None:
            raise Exception("Connection not established.")

        async with self._connection.cursor() as cursor:
            await cursor.execute(sql_command)
            columns = [desc[0] for desc in cursor.description]
            result = await cursor.fetchall()
            self._dataframe = pd.DataFrame(result, columns=columns)
            if index_column and index_column in self._dataframe.columns:
                self._dataframe.set


## ✨ Step 6: Integrating Async Support into DatabaseConnection Class {#-step-6-
integrating-async-support-into-databaseconnection-class}

Now that we have defined the PSYCOPG3 dictionary with async extensions, we need to
update the DatabaseConnection class to support asynchronous operations. Based on
the provided `connection.py`, here's how to enhance the existing
DatabaseConnection class:

1. **Update the Constructor** to support the async mode:

```python
def __init__(self, target_database_engine_type: TargetDatabaseEngine,
             target_dbconnect_option: TargetDatabaseConnectionStringOption,
             key_management: Optional[str] = None,
             use_async: bool = False):
    # Existing initialization code...

    # Add async mode flag
    self._use_async = use_async

    # Initialize appropriate transaction manager based on async mode
    if self._use_async and self._target_database_engine_type_name == "PSYCOPG3":
        self.transaction_manager = AsyncTransactionManager(self)
```
```

```
    else:
        self.transaction_manager = TransactionManager(self)
```

2. **Add Asynchronous Connection Method**:

```
async def connect_to_database_async(self) -> 'DatabaseConnection':
    """
    Establishes an asynchronous connection to the database using the finalized
connection parameters.

    This method retrieves the connection string from the database engine
configuration and
    attempts to connect to the database asynchronously. If the connection is
successful,
    it returns the current instance; otherwise, it raises a ConnectionError.

    Returns:
        DatabaseConnection: The current instance after successfully establishing a
connection.

    Raises:
        ConnectionError: If the connection to the database fails.
    """
    if not self._use_async:
        raise ValueError("Async mode is not enabled. Initialize with
use_async=True.")

    try:
        engine_module =
self.get_db_target_engine_module(self._get_async_module_name())
        async_connection_func = self._get_async_connection_func()
        self._connection = await async_connection_func()
        return self
    except Exception as e:
        raise ConnectionError(f"Failed to connect to database asynchronously:
{e}")
```

3. **Add Helper Methods for Async Support**:

```
def _get_async_module_name(self) -> str:
    """
    Get the module name for asynchronous operations based on the target database
engine.

    Returns:
        str: The module name for async operations.

    Raises:
        ValueError: If async mode is not supported for the selected database
```

```
engine.
    """
    if self._target_database_engine_type_name == "PSYCOPG3":
        return 'psycopg.AsyncConnection'
    # Add support for other engines as needed
    raise ValueError(f"Async mode not supported for
{self._target_database_engine_type_name}")

def _get_async_connection_func(self) -> Callable:
    """
    Get the async connection function for the target database engine.

    Returns:
        Callable: A function that creates an async connection when called.

    Raises:
        ValueError: If async mode is not supported for the selected database
engine.
    """
    engine_config = self._get_engine_config()
    if self._target_database_engine_type_name == "PSYCOPG3":
        return engine_config["ASYNC_EXTENSIONS"]
["LAMBDA_FINALIZED_PARAMETER_ASYNC_CONNECTION_STRING"](
            self._finalization_connection_parameters_string
        )
    # Add support for other engines as needed
    raise ValueError(f"Async mode not supported for
{self._target_database_engine_type_name}")
```

4. **Add Asynchronous Query Execution Methods**:

```
async def execute_async(self, sql_command: str, parameters: Optional[List] = None)
-> 'DatabaseConnection':
    """
    Executes a SQL command asynchronously.

    Args:
        sql_command (str): The SQL command to execute.
        parameters (Optional[List], optional): Parameters for the SQL command.
Defaults to None.

    Returns:
        DatabaseConnection: The current instance for method chaining.

    Raises:
        Exception: If the database connection has not been established.
    """
    if self._connection is None:
        raise Exception("Connection not established.")

    async with self._connection.cursor() as cursor:
        await cursor.execute(sql_command, parameters)
```

```
            self.rowcount = cursor.rowcount
        return self

    async def execute_sql_to_dataframe_async(self, sql_command: str, index_column: str
    = None) -> 'DatabaseConnection':
        """
        Executes a SQL command asynchronously and stores the result in a dataframe.

        Args:
            sql_command (str): The SQL command to execute.
            index_column (str, optional): The column to use as the index. Defaults to
    None.

        Returns:
            DatabaseConnection: The current instance for method chaining.

        Raises:
            Exception: If the database connection has not been established.
        """
        if self._connection is None:
            raise Exception("Connection not established.")

        async with self._connection.cursor() as cursor:
            await cursor.execute(sql_command)
            columns = [desc[0] for desc in cursor.description]
            result = await cursor.fetchall()
            self._dataframe = pd.DataFrame(result, columns=columns)
            if index_column and index_column in self._dataframe.columns:
                self._dataframe.set_index(index_column, inplace=True)
        return self

    async def execute_sql_to_prettytable_async(self, sql_command: str, is_print: bool
    = True) -> Optional[PrettyTable]:
        """
        Executes a SQL command asynchronously and formats the results as a
    PrettyTable.

        Args:
            sql_command (str): The SQL command to execute.
            is_print (bool, optional): Whether to print the table. Defaults to True.

        Returns:
            Optional[PrettyTable]: The PrettyTable object if is_print is False, None
    otherwise.

        Raises:
            Exception: If the database connection has not been established.
        """
        if self._connection is None:
            raise Exception("Connection not established.")

        async with self._connection.cursor() as cursor:
            await cursor.execute(sql_command)
            columns = [desc[0] for desc in cursor.description]
```

```
        result = await cursor.fetchall()

        table = PrettyTable()
        table.field_names = columns
        for row in result:
            table.add_row(row)

        if is_print:
            print(table)
            return None
        else:
            return table
```

5. **Add Asynchronous Close Method**:

```
async def close_async(self) -> 'DatabaseConnection':
    """
    Closes the database connection asynchronously.

    Returns:
        DatabaseConnection: The current instance for method chaining.
    """
    if self._connection:
        await self._connection.close()
        self._connection = None
    return self
```

6. **Add AsyncTransactionManager Class**:

```
class AsyncTransactionManager:
    """
    Transaction manager for asynchronous database operations.

    This class provides methods for managing transactions asynchronously,
    including beginning a transaction, committing changes, and performing
rollbacks.
    """

    def __init__(self, db_connector):
        """
        Initialize the AsyncTransactionManager.

        Args:
            db_connector: The database connector instance.
        """
        self.db_connector = db_connector
        self.transaction = None

    async def begin_transaction_async(self, isolation_level):
```

```
        """
        Begin a new transaction asynchronously with the specified isolation level.

        Args:
            isolation_level: The isolation level for the transaction.

        Returns:
            AsyncTransactionManager: The current instance for method chaining.
        """
        isolation_level_cmd = f"SET TRANSACTION ISOLATION LEVEL
{isolation_level.value}"
        async with self.db_connector._connection.cursor() as cursor:
            await cursor.execute(isolation_level_cmd)

        self.transaction = self.db_connector._connection.transaction()
        await self.transaction.start()
        return self

    async def commit_async(self):
        """
        Commit the current transaction asynchronously.

        Returns:
            AsyncTransactionManager: The current instance for method chaining.
        """
        if self.transaction:
            await self.transaction.commit()
            self.transaction = None
        return self

    async def rollback_async(self):
        """
        Rollback the current transaction asynchronously.

        Returns:
            AsyncTransactionManager: The current instance for method chaining.
        """
        if self.transaction:
            await self.transaction.rollback()
            self.transaction = None
        return self
```

7. **Update TargetDatabaseEngine Enum**:

Add PSYCOPG3 to the list of supported database engines in the `TargetDatabaseEngine` enum in `templates/constraints.py`:

```
class TargetDatabaseEngine(Enum):
    """
    Enum representing the target database engine types.
    """
```

```
    MSSQL = 1
    PSYCOPG2 = 2
    PSYCOPG3 = 3
    # Add other database engines as needed
```

These changes allow the DatabaseConnection class to support both synchronous and asynchronous operations with psycopg3, while maintaining backward compatibility with existing code.

Back to Table of Contents

📊 Step 7: Migrating SQL Syntax for PostgreSQL {#-step-7-migrating-sql-syntax-for-postgresql}

When migrating from SQL Server to PostgreSQL, several SQL syntax changes are necessary. Here's a guide to common migrations you'll need to make:

Date and Time Functions

SQL Server	PostgreSQL	Notes
`SYSDATETIME()`	`CURRENT_TIMESTAMP` or `now()`	Current date and time
`GETDATE()`	`CURRENT_DATE`	Current date only
`DATEADD(day, 7, date)`	`date + INTERVAL '7 days'`	Adding intervals
`DATEADD(month, 3, date)`	`date + INTERVAL '3 months'`	Adding months
`DATEDIFF(day, date1, date2)`	`date2 - date1`	Result in days

Identity and Sequence Functions

SQL Server	PostgreSQL	Notes
`IDENT_CURRENT('table')`	`currval(pg_get_serial_sequence('schema.table', 'column'))`	Get last identity value
`SCOPE_IDENTITY()`	`lastval()`	Get last generated value in session
`IDENTITY(INT, 1, 1)`	`SERIAL` or `INT GENERATED BY DEFAULT AS IDENTITY`	Define auto-incrementing column

Syntax Differences

SQL Server	PostgreSQL	Notes
`[schema].[table]`	`"schema"."table"`	Identifier quoting

SQL Server	PostgreSQL	Notes
`?`	`%s`	Parameter placeholder
`SELECT TOP 10`	`SELECT ... LIMIT 10`	Result limiting
`WITH NOLOCK`	`SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED`	Isolation level
`ISNULL(col, default)`	`COALESCE(col, default)`	Null handling

Data Type Differences

SQL Server	PostgreSQL	Notes
`NVARCHAR(n)`	`VARCHAR(n)`	Unicode string
`VARCHAR(MAX)`	`TEXT`	Variable-length string
`DATETIME`	`TIMESTAMP`	Date and time
`DATETIME2`	`TIMESTAMP`	Higher precision date/time
`BIT`	`BOOLEAN`	Boolean value
`UNIQUEIDENTIFIER`	`UUID`	Unique identifier

Example SQL Conversion

SQL Server Query:

```
SELECT TOP 10
    e.[EmployeeID],
    e.[FirstName] + ' ' + e.[LastName] AS [FullName],
    CONVERT(VARCHAR, e.[HireDate], 101) AS [HireDate],
    DATEDIFF(YEAR, e.[HireDate], GETDATE()) AS [YearsEmployed]
FROM [HumanResources].[Employee] e
WHERE e.[HireDate] >= DATEADD(YEAR, -5, GETDATE())
ORDER BY e.[HireDate] DESC;
```

PostgreSQL Equivalent:

```
SELECT
    e."EmployeeID",
    e."FirstName" || ' ' || e."LastName" AS "FullName",
    TO_CHAR(e."HireDate", 'MM/DD/YYYY') AS "HireDate",
    EXTRACT(YEAR FROM AGE(CURRENT_DATE, e."HireDate")) AS "YearsEmployed"
FROM "HumanResources"."Employee" e
WHERE e."HireDate" >= (CURRENT_DATE - INTERVAL '5 years')
```

```
ORDER BY e."HireDate" DESC
LIMIT 10;
```

These syntax changes must be applied when migrating SQL queries from SQL Server to PostgreSQL. The most common adjustments involve date/time functions, sequence handling, and identifier quoting.

Back to Table of Contents

Step 8: Example Implementation {#-step-8-example-implementation}

Let's put everything together with complete examples that demonstrate both synchronous and asynchronous operations using the migrated configuration.

Synchronous Database Operations

```
from pydatabaseconnection.connection import DatabaseConnection
from pydatabaseconnection.db_constants.enum_connection_constants import
TargetDatabaseEngine, TargetDatabaseConnectionStringOption
from pydatabaseconnection.templates.constraints import IsolationLevel

def demonstrate_basic_transaction():
    """Demonstrate basic transaction operations with explicit transaction
control."""
    print("\n=== Starting Basic Transaction Demo ===")

    db = DatabaseConnection(
        target_database_engine_type=TargetDatabaseEngine.PSYCOPG3,

target_dbconnect_option=TargetDatabaseConnectionStringOption.DefaultFormatAndDefau
ltUserPwd,
    )

    try:
        # Connect to database
        db = (
            db.set_credentials_based_on_connect_option()
            .finalization_of_connection_parameters(
                SERVER="127.0.0.1",
                PORT_NUMBER=5432,
                DATABASE_NAME="AdventureWorks",
                USERNAME="postgres",
                PASSWORD="postgres",
            )
            .connect_to_database()
        )

        print("Connected to AdventureWorks database")

        # Begin transaction
        db.transaction_manager.begin_transaction(IsolationLevel.READ_COMMITTED)
```

```
        try:
            # Execute queries within transaction
            db.execute("INSERT INTO \"Production\".\"Category\" (\"Name\") VALUES
(%s)", ["Test Category"])

            # Get the inserted ID
            db.execute_sql_to_dataframe(
                "SELECT
currval(pg_get_serial_sequence('\"Production\".\"Category\"', '\"CategoryID\"'))
AS \"CategoryID\""
            )
            category_id = db.to_dataframe()["CategoryID"].iloc[0]
            print(f"Inserted category ID: {category_id}")

            # Commit transaction
            db.transaction_manager.commit()
            print("Transaction committed successfully")

        except Exception as e:
            # Rollback on error
            db.transaction_manager.rollback()
            print(f"Transaction rolled back: {str(e)}")
            raise
    finally:
        db.close()
        print("=== Transaction Demo Completed ===")

if __name__ == "__main__":
    demonstrate_basic_transaction()
```

Asynchronous Database Operations

```
import asyncio
from pydatabaseconnection.connection import DatabaseConnection
from pydatabaseconnection.db_constants.enum_connection_constants import
TargetDatabaseEngine, TargetDatabaseConnectionStringOption
from pydatabaseconnection.templates.constraints import IsolationLevel

async def demonstrate_basic_transaction_async():
    """Demonstrate basic transaction operations with explicit transaction control
asynchronously."""
    print("\n=== Starting Async Transaction Demo ===")

    db = DatabaseConnection(
        target_database_engine_type=TargetDatabaseEngine.PSYCOPG3,

target_dbconnect_option=TargetDatabaseConnectionStringOption.DefaultFormatAndDefau
ltUserPwd,
        use_async=True  # Enable async mode
    )
```

```
    try:
        # Connect to database asynchronously
        db = (
            db.set_credentials_based_on_connect_option()
            .finalization_of_connection_parameters(
                SERVER="127.0.0.1",
                PORT_NUMBER=5432,
                DATABASE_NAME="AdventureWorks",
                USERNAME="postgres",
                PASSWORD="postgres",
            )
        )

        await db.connect_to_database_async()
        print("Connected to AdventureWorks database")

        # Begin transaction asynchronously
        await
db.transaction_manager.begin_transaction_async(IsolationLevel.READ_COMMITTED)

        try:
            # Execute queries within transaction asynchronously
            await db.execute_async(
                "INSERT INTO \"Production\".\"Category\" (\"Name\") VALUES (%s)",
                ["Async Test Category"]
            )

            # Get the inserted ID asynchronously
            await db.execute_sql_to_dataframe_async(
                "SELECT
currval(pg_get_serial_sequence('\"Production\".\"Category\"', '\"CategoryID\"'))
AS \"CategoryID\""
            )
            category_id = db.to_dataframe()["CategoryID"].iloc[0]
            print(f"Inserted category ID: {category_id}")

            # Commit transaction asynchronously
            await db.transaction_manager.commit_async()
            print("Transaction committed successfully")

        except Exception as e:
            # Rollback on error asynchronously
            await db.transaction_manager.rollback_async()
            print(f"Transaction rolled back: {str(e)}")
            raise
    finally:
        await db.close_async()
        print("=== Async Transaction Demo Completed ===")

if __name__ == "__main__":
    asyncio.run(demonstrate_basic_transaction_async())
```

Connection Pool Example

```
import asyncio
from pydatabaseconnection.connection import DatabaseConnection
from pydatabaseconnection.db_constants.enum_connection_constants import
TargetDatabaseEngine, TargetDatabaseConnectionStringOption

async def demonstrate_connection_pool():
    """Demonstrate connection pool usage with asynchronous operations."""
    print("\n=== Starting Connection Pool Demo ===")

    # Create database connection
    db = DatabaseConnection(
        target_database_engine_type=TargetDatabaseEngine.PSYCOPG3,

target_dbconnect_option=TargetDatabaseConnectionStringOption.DefaultFormatAndDefau
ltUserPwd,
        use_async=True
    )

    # Set connection parameters
    db = (
        db.set_credentials_based_on_connect_option()
        .finalization_of_connection_parameters(
            SERVER="127.0.0.1",
            PORT_NUMBER=5432,
            DATABASE_NAME="AdventureWorks",
            USERNAME="postgres",
            PASSWORD="postgres",
        )
    )

    # Get the connection string
    conn_string = db._finalization_connection_parameters_string

    # Create connection pool
    engine_config = db._get_engine_config()
    pool_creator = engine_config["ASYNC_EXTENSIONS"]["CONNECTION_POOL"]
["LAMBDA_CREATE_POOL"](
        conn_string, min_size=2, max_size=10
    )
    pool = await pool_creator()

    print("Connection pool created")

    try:
        # Perform concurrent database operations
        async def query_categories(pool, category_id):
            async with pool.connection() as conn:
                async with conn.cursor() as cursor:
                    await cursor.execute(
                        "SELECT \"CategoryID\", \"Name\" FROM
```

```
\"Production\".\"Category\" WHERE \"CategoryID\" = %s",
                        [category_id]
                    )
                    result = await cursor.fetchone()
                    return result

        # Execute concurrent queries
        tasks = [query_categories(pool, i) for i in range(1, 6)]
        results = await asyncio.gather(*tasks)

        # Print results
        print("\nResults from concurrent queries:")
        for result in results:
            if result:
                print(f"Category ID: {result[0]}, Name: {result[1]}")
            else:
                print("Category not found")

    finally:
        # Close the pool
        await pool.close()
        print("\n=== Connection Pool Demo Completed ===")

if __name__ == "__main__":
    asyncio.run(demonstrate_connection_pool())
```

These examples demonstrate how to use both synchronous and asynchronous operations with psycopg3 after the migration, showcasing the power and flexibility of the new implementation.

Back to Table of Contents

📌 Conclusion {#-conclusion}

In this comprehensive guide, we've walked through the complete process of migrating from psycopg2 to psycopg3, with a special focus on leveraging the new asynchronous capabilities. By following these steps, you've enhanced your DatabaseConnection class to support both traditional synchronous operations and modern asynchronous workflows.

Key accomplishments:

1. **Created a Complete PSYCOPG3 Dictionary Structure**: Defined a robust configuration that supports all psycopg3 features
2. **Added Asynchronous Extensions**: Integrated native async support for non-blocking database operations
3. **Implemented Connection Pooling**: Added efficient connection management for high-concurrency scenarios
4. **Extended DatabaseConnection Class**: Updated the existing class to support both sync and async operations
5. **Added Transaction Management**: Created transaction managers for both operation modes
6. **Migrated SQL Syntax**: Provided guidance for adapting SQL queries for PostgreSQL

7. **Developed Example Implementations**: Created practical examples showcasing the new capabilities

The benefits of this migration include:

- **Improved Performance**: Non-blocking async operations and connection pooling enhance throughput
- **Better Resource Utilization**: More efficient handling of database connections
- **Enhanced Developer Experience**: Modern async/await patterns improve code readability
- **Future-Proofing**: Alignment with modern Python practices and PostgreSQL features
- **Backward Compatibility**: Maintained support for existing synchronous code

By completing this migration, you've brought your database layer into the modern era of Python and PostgreSQL development, setting a solid foundation for building high-performance applications.

Back to Table of Contents

📋 Complete PSYCOPG3 Dictionary Reference {#-complete-psycopg3-dictionary-reference}

For reference, here's the complete PSYCOPG3 dictionary that you should add to your `database_connection_constants.py` file:

```
PSYCOPG3: Dict[str, Dict[str, Any]] = {
    "PSYCOPG3": {
        "DEFAULT_PARAMETERS": {
            "DATABASE_NAME": "NorthwindOrderSubsystem",
            "SERVER": "localhost",
            "PORT_NUMBER": 5432,  # Default PostgreSQL port
            "USERNAME": "postgres",
            "PASSWORD": "Password",
            "FORMAT_CONNECTION_STRING": {
                "LOAD_DEFAULT_PARAMETER_KEYS":
["DATABASE_NAME","SERVER","PORT_NUMBER"],
                "METHOD_LOAD_DEFAULT_PARAMETER_KEYS_DICT":
load_default_parameter_keys_dict,
                "IS_DSN_NAME_SUPPORTED": "N",
                "BASE_CONNECTION_STRING": "host={0} port={1} dbname={2} user={3}
password={4}",
                "BASE_ODBC_DSN_CONNECTION_STRING": None
            }
        },
        "FINALIZED_CONNECTION_PARAMS": {
            "LAMBDA_FINALIZED_BASE_CONNECTION_STRING": lambda d: d["PSYCOPG3"]
["DEFAULT_PARAMETERS"]["FORMAT_CONNECTION_STRING"]
["BASE_CONNECTION_STRING"].format(
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["SERVER"],
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["PORT_NUMBER"],
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["DATABASE_NAME"],
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["USERNAME"],
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["PASSWORD"]
            ),
            "LAMBDA_FINALIZED_DSN_CONNECTION_STRING": lambda d: d["PSYCOPG3"]
["DEFAULT_PARAMETERS"]["FORMAT_CONNECTION_STRING"]
```

```
["BASE_ODBC_DSN_CONNECTION_STRING"].format(
                d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["DSN_NAME"]
            ) if d["PSYCOPG3"]["DEFAULT_PARAMETERS"]["FORMAT_CONNECTION_STRING"]
["IS_DSN_NAME_SUPPORTED"] == "Y" else None
        },
        "CRUDBASIC": {
            "IMPORT": "psycopg",
            "PARAMETER_PLACEHOLDER": "%s",
            "IS_USERNAME_AND_PASSWORD_REQUIRED": True
        },
        "CREATE_DATABASE_CONNECTION": {
            "CONNECTION_NOTES": "Use psycopg.connect with the appropriate
connection string for synchronous operations.",
            "LAMBDA_FINALIZED_PARAMETER_CONNECTION_STRING": lambda
finalized_parameter_connection_string: lambda:
get_db_target_engine_module('psycopg').connect(finalized_parameter_connection_stri
ng)
        },
        "ASYNC_EXTENSIONS": {
            "IMPORT": "psycopg.AsyncConnection",
            "IS_SUPPORTED": "Y",
            "CONNECTION_NOTES": "Use psycopg.AsyncConnection.connect with the
appropriate connection string for asynchronous operations.",
            "LAMBDA_FINALIZED_PARAMETER_ASYNC_CONNECTION_STRING": lambda
finalized_parameter_connection_string: lambda:
get_db_target_engine_module('psycopg.AsyncConnection').connect(finalized_parameter
_connection_string),
            "ASYNC_CURSOR": {
                "IMPORT": "psycopg.AsyncCursor",
                "USAGE_NOTES": "Use 'async with conn.cursor() as cur:' pattern for
cursor operations."
            },
            "ASYNC_EXECUTE": {
                "EXECUTE_PATTERN": "await cursor.execute(query, params)",
                "FETCH_PATTERN": {
                    "FETCHONE": "await cursor.fetchone()",
                    "FETCHMANY": "await cursor.fetchmany(size)",
                    "FETCHALL": "await cursor.fetchall()"
                }
            },
            "ASYNC_TRANSACTION": {
                "BEGIN_PATTERN": "async with conn.transaction():",
                "USAGE_NOTES": "Transactions are managed using context managers in
psycopg3."
            },
            "CONNECTION_POOL": {
                "IMPORT": "psycopg_pool.AsyncConnectionPool",
                "USAGE_NOTES": "Use AsyncConnectionPool for connection pooling
with async operations.",
                "LAMBDA_CREATE_POOL": lambda
finalized_parameter_connection_string, min_size=1, max_size=10: lambda:
get_db_target_engine_module('psycopg_pool').AsyncConnectionPool(finalized_paramete
r_connection_string, min_size=min_size, max_size=max_size)
            }
```

```
        },
        "DATABASE_CONNECTOR_EXTENSIONS": {
            "IMPORT_DEPENDENCIES": [
                "asyncio",
                "pandas as pd",
                "prettytable",
                "typing import Dict, Any, Optional, Union, List, Tuple"
            ],
            "INITIALIZATION": {
                "CONSTRUCTOR_PARAMS": {
                    "use_async": {
                        "TYPE": "bool",
                        "DEFAULT": "False",
                        "DESCRIPTION": "Flag to determine if async mode should be
used"
                    }
                },
                "TRANSACTION_MANAGER_INIT": "self.transaction_manager =
AsyncTransactionManager(self) if use_async else TransactionManager(self)"
            },
            "SYNC_METHODS": {
                "connect_to_database": {
                    "CODE": """
                    engine_module =
get_db_target_engine_module(self._get_module_name())
                    connection_func = self._get_connection_func()
                    self.connection = connection_func()
                    return self
                    """,
                    "DESCRIPTION": "Create synchronous database connection"
                },
                "_get_module_name": {
                    "CODE": """
                    if self.target_database_engine ==
TargetDatabaseEngine.PSYCOPG3:
                        return 'psycopg'
                    # Add support for other engines
                    raise ValueError(f"Unsupported database engine:
{self.target_database_engine}")
                    """,
                    "DESCRIPTION": "Get the module name for the selected database
engine"
                },
                "_get_connection_func": {
                    "CODE": """
                    if self.target_database_engine ==
TargetDatabaseEngine.PSYCOPG3:
                        conn_string = self._get_connection_string()
                        return PSYCOPG3["PSYCOPG3"]["CREATE_DATABASE_CONNECTION"]
["LAMBDA_FINALIZED_PARAMETER_CONNECTION_STRING"](conn_string)
                    # Add support for other engines
                    raise ValueError(f"Unsupported database engine:
{self.target_database_engine}")
                    """,
```

```
                    "DESCRIPTION": "Get the connection function for the selected
database engine"
                },
                "execute": {
                    "CODE": """
                    with self.connection.cursor() as cursor:
                        cursor.execute(query, parameters)
                        self.rowcount = cursor.rowcount
                    return self
                    """,
                    "DESCRIPTION": "Execute a SQL query with parameters"
                },
                "execute_query_to_dataframe": {
                    "CODE": """
                    with self.connection.cursor() as cursor:
                        cursor.execute(query, parameters)
                        columns = [desc[0] for desc in cursor.description]
                        result = cursor.fetchall()
                        self.dataframe = pd.DataFrame(result, columns=columns)
                    return self
                    """,
                    "DESCRIPTION": "Execute a SQL query and store results in a
dataframe"
                },
                "execute_query_to_prettytable": {
                    "CODE": """
                    with self.connection.cursor() as cursor:
                        cursor.execute(query, parameters)
                        columns = [desc[0] for desc in cursor.description]
                        result = cursor.fetchall()

                        pt = prettytable.PrettyTable()
                        pt.field_names = columns
                        for row in result:
                            pt.add_row(row)

                        print(pt)
                        self.prettytable = pt
                    return self
                    """,
                    "DESCRIPTION": "Execute a SQL query and display results in a
prettytable"
                },
                "to_dataframe": {
                    "CODE": """
                    return self.dataframe
                    """,
                    "DESCRIPTION": "Return the current dataframe"
                },
                "close": {
                    "CODE": """
                    if self.connection:
                        self.connection.close()
                        self.connection = None
```

```
                    return self
                    """,
                    "DESCRIPTION": "Close the database connection"
                }
            },
            "ASYNC_METHODS": {
                "connect_to_database_async": {
                    "CODE": """
                    engine_module =
get_db_target_engine_module(self._get_async_module_name())
                    connection_func = self._get_async_connection_func()
                    self.connection = await connection_func()
                    return self
                    """,
                    "DESCRIPTION": "Create asynchronous database connection"
                },
                "_get_async_module_name": {
                    "CODE": """
                    if self.target_database_engine ==
TargetDatabaseEngine.PSYCOPG3:
                        return 'psycopg.AsyncConnection'
                    # Add support for other engines
                    raise ValueError(f"Async mode not supported for
{self.target_database_engine}")
                    """,
                    "DESCRIPTION": "Get the async module name for the selected
database engine"
                },
                "_get_async_connection_func": {
                    "CODE": """
                    if self.target_database_engine ==
TargetDatabaseEngine.PSYCOPG3:
                        conn_string = self._get_connection_string()
                        return PSYCOPG3["PSYCOPG3"]["ASYNC_EXTENSIONS"]
["LAMBDA_FINALIZED_PARAMETER_ASYNC_CONNECTION_STRING"](conn_string)
                    # Add support for other engines
                    raise ValueError(f"Async mode not supported for
{self.target_database_engine}")
                    """,
                    "DESCRIPTION": "Get the async connection function for the
selected database engine"
                },
                "execute_async": {
                    "CODE": """
                    async with self.connection.cursor() as cursor:
                        await cursor.execute(query, parameters)
                        self.rowcount = cursor.rowcount
                    return self
                    """,
                    "DESCRIPTION": "Execute a SQL query asynchronously with
parameters"
                },
                "execute_query_to_dataframe_async": {
                    "CODE": """
```

```
                    async with self.connection.cursor() as cursor:
                        await cursor.execute(query, parameters)
                        columns = [desc[0] for desc in cursor.description]
                        result = await cursor.fetchall()
                        self.dataframe = pd.DataFrame(result, columns=columns)
                    return self
                    """,
                    "DESCRIPTION": "Execute a SQL query asynchronously and store
results in a dataframe"
                },
                "execute_query_to_prettytable_async": {
                    "CODE": """
                    async with self.connection.cursor() as cursor:
                        await cursor.execute(query, parameters)
                        columns = [desc[0] for desc in cursor.description]
                        result = await cursor.fetchall()

                        pt = prettytable.PrettyTable()
                        pt.field_names = columns
                        for row in result:
                            pt.add_row(row)

                        print(pt)
                        self.prettytable = pt
                    return self
                    """,
                    "DESCRIPTION": "Execute a SQL query asynchronously and display
results in a prettytable"
                },
                "close_async": {
                    "CODE": """
                    if self.connection:
                        await self.connection.close()
                        self.connection = None
                    return self
                    """,
                    "DESCRIPTION": "Close the database connection asynchronously"
                }
            }
        },
        "TRANSACTION_MANAGER_EXTENSIONS": {
            "SYNC_TRANSACTION_MANAGER": {
                "INITIALIZATION": {
                    "CODE": """
                    def __init__(self, db_connector):
                        self.db_connector = db_connector
                        self.transaction_active = False
                    """
                },
                "METHODS": {
                    "begin_transaction": {
                        "CODE": """
                        if self.transaction_active:
                            raise ValueError("Transaction already in progress")
```

```
                        # Set isolation level
                        isolation_level_cmd = f"SET TRANSACTION ISOLATION LEVEL
{isolation_level.value}"
                        with self.db_connector.connection.cursor() as cursor:
                            cursor.execute(isolation_level_cmd)

                        # Begin transaction
                        with self.db_connector.connection.cursor() as cursor:
                            cursor.execute("BEGIN")

                        self.transaction_active = True
                        return self
                        """,
                        "DESCRIPTION": "Begin a new transaction with specified
isolation level"
                    },
                    "commit": {
                        "CODE": """
                        if not self.transaction_active:
                            raise ValueError("No transaction in progress")

                        with self.db_connector.connection.cursor() as cursor:
                            cursor.execute("COMMIT")

                        self.transaction_active = False
                        return self
                        """,
                        "DESCRIPTION": "Commit the current transaction"
                    },
                    "rollback": {
                        "CODE": """
                        if not self.transaction_active:
                            raise ValueError("No transaction in progress")

                        with self.db_connector.connection.cursor() as cursor:
                            cursor.execute("ROLLBACK")

                        self.transaction_active = False
                        return self
                        """,
                        "DESCRIPTION": "Rollback the current transaction"
                    }
                }
            },
            "ASYNC_TRANSACTION_MANAGER": {
                "INITIALIZATION": {
                    "CODE": """
                    def __init__(self, db_connector):
                        self.db_connector = db_connector
                        self.transaction = None
                    """
                },
                "METHODS": {
```

```
                    "begin_transaction_async": {
                        "CODE": """
                        # Set isolation level
                        isolation_level_cmd = f"SET TRANSACTION ISOLATION LEVEL
{isolation_level.value}"
                        async with self.db_connector.connection.cursor() as
cursor:
                            await cursor.execute(isolation_level_cmd)

                        # Begin transaction context
                        self.transaction =
self.db_connector.connection.transaction()
                        await self.transaction.start()
                        return self
                        """,
                        "DESCRIPTION": "Begin a new transaction asynchronously
with specified isolation level"
                    },
                    "commit_async": {
                        "CODE": """
                        if self.transaction:
                            await self.transaction.commit()
                            self.transaction = None
                        return self
                        """,
                        "DESCRIPTION": "Commit the current transaction
asynchronously"
                    },
                    "rollback_async": {
                        "CODE": """
                        if self.transaction:
                            await self.transaction.rollback()
                            self.transaction = None
                        return self
                        """,
                        "DESCRIPTION": "Rollback the current transaction
asynchronously"
                    }
                }
            }
        }
    }
}
```

🚀 Example: Migrating the AdventureWorks Database Example {#-example-migrating-the-adventureworks-database-example}

Let's implement a complete example that demonstrates migrating the provided SQL Server code to work with PostgreSQL and psycopg3 with async capabilities. This example will showcase all the key aspects of the migration:

Original SQL Server Code:

```
def demonstrate_basic_transaction():
    """Demonstrate basic transaction operations with explicit transaction
control."""
    print("\n=== Starting Basic Transaction Demo ===")

    # Constants for the transaction
    PRODUCT_ID = 707
    LOCATION_ID = 7
    ORDER_QTY = 5

    db = DatabaseConnector(
        target_database_engine=TargetDatabaseEngine.MSSQL,

target_database_engine_connect_option=TargetDatabaseConnectionStringOption.Default
FormatAndDefaultUserPwd,
    )

    try:
        # Connect to database
        db = (
            db.set_credentials_based_on_connect_option()
            .finalization_of_connection_parameters(
                DRIVER="{ODBC Driver 17 for SQL Server}",
                SERVER="127.0.0.1",
                PORT_NUMBER=13001,
                DATABASE_NAME="AdventureWorks2019",
            )
            .connect_to_database()
        )

        print("Connected to AdventureWorks2019 database")

        # First verify product exists and get details
        product_check_query = """
        SELECT p.ProductID,
               p.Name AS ProductName,
               pi.LocationID,
               pi.Quantity AS CurrentStock
        FROM Production.Product p
        INNER JOIN Production.ProductInventory pi
            ON p.ProductID = pi.ProductID
        WHERE p.ProductID = ? AND pi.LocationID = ?
        """

        print("\nVerifying product exists:")
        sql_parameters = (PRODUCT_ID, LOCATION_ID)
        db.execute_query_to_prettytable(product_check_query, sql_parameters)

        # Explicitly begin transaction
        print("\nExplicitly beginning transaction...")
        db.transaction_manager.begin_transaction(IsolationLevel.READ_COMMITTED)
```

```
        try:
            # 1. Insert a new work order
            work_order_insert = """
            INSERT INTO Production.WorkOrder
                (ProductID, OrderQty, ScrappedQty, StartDate, DueDate,
ModifiedDate)
            VALUES
                (?, ?, 0, SYSDATETIME(), DATEADD(day, 7, SYSDATETIME()),
SYSDATETIME())
            """
            sql_parameters = (PRODUCT_ID, ORDER_QTY)
            db.execute(work_order_insert, sql_parameters)

            # Get the ID using a separate query
            get_id_query = "SELECT IDENT_CURRENT('Production.WorkOrder') AS
WorkOrderID"
            db.execute_query_to_dataframe(get_id_query)
            df = db.to_dataframe()
            work_order_id = int(df["WorkOrderID"].iloc[0])
            print(f"\nCreated Work Order ID: {work_order_id}")

            # 2. Insert work order routing
            routing_query = """
            INSERT INTO Production.WorkOrderRouting
                (WorkOrderID, ProductID, OperationSequence, LocationID,
                 ScheduledStartDate, ScheduledEndDate, ModifiedDate, PlannedCost)
            VALUES
                (?, ?, 1, ?, SYSDATETIME(),
                 DATEADD(day, 7, SYSDATETIME()), SYSDATETIME(), 100.00)
            """
            sql_parameters = (work_order_id, PRODUCT_ID, LOCATION_ID)
            db.execute(routing_query, sql_parameters)
            print(f"Created Work Order Routing for Order {work_order_id}")

            # Commit transaction
            print("\nExplicitly committing transaction...")
            db.transaction_manager.commit()
            print("Transaction committed successfully!")

        except Exception as e:
            # Explicitly rollback transaction
            print(f"\nError occurred: {str(e)}")
            print("Explicitly rolling back transaction...")
            db.transaction_manager.rollback()
            raise

    except Exception as e:
        print(f"\nOuter error occurred: {str(e)}")
        raise
    finally:
        db.close()
        print("\n=== Transaction Demo Completed ===")
```

Migrated PostgreSQL Code with psycopg3 (Synchronous Version):

```
from pydatabaseconnection.connection import DatabaseConnection
from pydatabaseconnection.db_constants.enum_connection_constants import
TargetDatabaseEngine, TargetDatabaseConnectionStringOption
from pydatabaseconnection.templates.constraints import IsolationLevel

def demonstrate_basic_transaction():
    """Demonstrate basic transaction operations with explicit transaction control
using PostgreSQL."""
    print("\n=== Starting Basic Transaction Demo ===")

    # Constants for the transaction
    PRODUCT_ID = 707
    LOCATION_ID = 7
    ORDER_QTY = 5

    db = DatabaseConnection(
        target_database_engine_type=TargetDatabaseEngine.PSYCOPG3,  # Updated to
use PSYCOPG3

target_dbconnect_option=TargetDatabaseConnectionStringOption.DefaultFormatAndDefau
ltUserPwd,
    )

    try:
        # Connect to database with PostgreSQL parameters
        db = (
            db.set_credentials_based_on_connect_option()
            .finalization_of_connection_parameters(
                SERVER="127.0.0.1",
                PORT_NUMBER=5432,  # Standard PostgreSQL port
                DATABASE_NAME="AdventureWorks",  # PostgreSQL database name
            )
            .connect_to_database()
        )

        print("Connected to AdventureWorks database")

        # First verify product exists and get details
        product_check_query = """
        SELECT p."ProductID",
               p."Name" AS "ProductName",
               pi."LocationID",
               pi."Quantity" AS "CurrentStock"
        FROM "Production"."Product" p
        INNER JOIN "Production"."ProductInventory" pi
            ON p."ProductID" = pi."ProductID"
        WHERE p."ProductID" = %s AND pi."LocationID" = %s
        """

        print("\nVerifying product exists:")
```

```
        sql_parameters = (PRODUCT_ID, LOCATION_ID)
        db.execute_query_to_prettytable(product_check_query, sql_parameters)

        # Explicitly begin transaction
        print("\nExplicitly beginning transaction...")
        db.transaction_manager.begin_transaction(IsolationLevel.READ_COMMITTED)

        try:
            # 1. Insert a new work order
            work_order_insert = """
            INSERT INTO "Production"."WorkOrder"
                ("ProductID", "OrderQty", "ScrappedQty", "StartDate", "DueDate",
"ModifiedDate")
            VALUES
                (%s, %s, 0, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP + INTERVAL '7
days', CURRENT_TIMESTAMP)
            """
            sql_parameters = (PRODUCT_ID, ORDER_QTY)
            db.execute(work_order_insert, sql_parameters)

            # Get the ID using currval function for the sequence
            get_id_query = """
            SELECT currval(pg_get_serial_sequence('"Production"."WorkOrder"',
'"WorkOrderID"')) AS "WorkOrderID"
            """
            db.execute_query_to_dataframe(get_id_query)
            df = db.to_dataframe()
            work_order_id = int(df["WorkOrderID"].iloc[0])
            print(f"\nCreated Work Order ID: {work_order_id}")

            # 2. Insert work order routing
            routing_query = """
            INSERT INTO "Production"."WorkOrderRouting"
                ("WorkOrderID", "ProductID", "OperationSequence", "LocationID",
                 "ScheduledStartDate", "ScheduledEndDate", "ModifiedDate",
"PlannedCost")
            VALUES
                (%s, %s, 1, %s, CURRENT_TIMESTAMP,
                 CURRENT_TIMESTAMP + INTERVAL '7 days', CURRENT_TIMESTAMP, 100.00)
            """
            sql_parameters = (work_order_id, PRODUCT_ID, LOCATION_ID)
            db.execute(routing_query, sql_parameters)
            print(f"Created Work Order Routing for Order {work_order_id}")

            # Explicitly commit transaction
            print("\nExplicitly committing transaction...")
            db.transaction_manager.commit()
            print("Transaction committed successfully!")

        except Exception as e:
            # Explicitly rollback transaction
            print(f"\nError occurred: {str(e)}")
            print("Explicitly rolling back transaction...")
            db.transaction_manager.rollback()
```

```
            raise

    except Exception as e:
        print(f"\nOuter error occurred: {str(e)}")
        raise
    finally:
        db.close()
        print("\n=== Transaction Demo Completed ===")


if __name__ == "__main__":
    demonstrate_basic_transaction()
```

Migrated PostgreSQL Code with psycopg3 (Asynchronous Version):

```
import asyncio
from pydatabaseconnection.connection import DatabaseConnection
from pydatabaseconnection.db_constants.enum_connection_constants import
TargetDatabaseEngine, TargetDatabaseConnectionStringOption
from pydatabaseconnection.templates.constraints import IsolationLevel

async def demonstrate_basic_transaction_async():
    """Demonstrate basic transaction operations with explicit transaction control
using PostgreSQL asynchronously."""
    print("\n=== Starting Basic Async Transaction Demo ===")

    # Constants for the transaction
    PRODUCT_ID = 707
    LOCATION_ID = 7
    ORDER_QTY = 5

    db = DatabaseConnection(
        target_database_engine_type=TargetDatabaseEngine.PSYCOPG3,  # Using
PSYCOPG3

target_dbconnect_option=TargetDatabaseConnectionStringOption.DefaultFormatAndDefau
ltUserPwd,
        use_async=True  # Enable async mode
    )

    try:
        # Connect to database with PostgreSQL parameters asynchronously
        db = (
            db.set_credentials_based_on_connect_option()
            .finalization_of_connection_parameters(
                SERVER="127.0.0.1",
                PORT_NUMBER=5432,  # Standard PostgreSQL port
                DATABASE_NAME="AdventureWorks",  # PostgreSQL database name
            )
        )
```

```
        # Connect asynchronously
        await db.connect_to_database_async()
        print("Connected to AdventureWorks database")

        # First verify product exists and get details
        product_check_query = """
        SELECT p."ProductID",
               p."Name" AS "ProductName",
               pi."LocationID",
               pi."Quantity" AS "CurrentStock"
        FROM "Production"."Product" p
        INNER JOIN "Production"."ProductInventory" pi
            ON p."ProductID" = pi."ProductID"
        WHERE p."ProductID" = %s AND pi."LocationID" = %s
        """

        print("\nVerifying product exists:")
        sql_parameters = (PRODUCT_ID, LOCATION_ID)
        await db.execute_query_to_prettytable_async(product_check_query,
sql_parameters)

        # Explicitly begin transaction
        print("\nExplicitly beginning transaction...")
        await
db.transaction_manager.begin_transaction_async(IsolationLevel.READ_COMMITTED)

        try:
            # 1. Insert a new work order
            work_order_insert = """
            INSERT INTO "Production"."WorkOrder"
                ("ProductID", "OrderQty", "ScrappedQty", "StartDate", "DueDate",
"ModifiedDate")
            VALUES
                (%s, %s, 0, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP + INTERVAL '7
days', CURRENT_TIMESTAMP)
            """
            sql_parameters = (PRODUCT_ID, ORDER_QTY)
            await db.execute_async(work_order_insert, sql_parameters)

            # Get the ID using currval function for the sequence
            get_id_query = """
            SELECT currval(pg_get_serial_sequence('"Production"."WorkOrder"',
'"WorkOrderID"')) AS "WorkOrderID"
            """
            await db.execute_query_to_dataframe_async(get_id_query)
            df = db.to_dataframe()
            work_order_id = int(df["WorkOrderID"].iloc[0])
            print(f"\nCreated Work Order ID: {work_order_id}")

            # 2. Insert work order routing
            routing_query = """
            INSERT INTO "Production"."WorkOrderRouting"
                ("WorkOrderID", "ProductID", "OperationSequence", "LocationID",
                 "ScheduledStartDate", "ScheduledEndDate", "ModifiedDate",
```

```
"PlannedCost")
            VALUES
                (%s, %s, 1, %s, CURRENT_TIMESTAMP,
                 CURRENT_TIMESTAMP + INTERVAL '7 days', CURRENT_TIMESTAMP, 100.00)
            """
            sql_parameters = (work_order_id, PRODUCT_ID, LOCATION_ID)
            await db.execute_async(routing_query, sql_parameters)
            print(f"Created Work Order Routing for Order {work_order_id}")

            # More operations could be added here...

            # Explicitly commit transaction
            print("\nExplicitly committing transaction...")
            await db.transaction_manager.commit_async()
            print("Transaction committed successfully!")

        except Exception as e:
            # Explicitly rollback transaction
            print(f"\nError occurred: {str(e)}")
            print("Explicitly rolling back transaction...")
            await db.transaction_manager.rollback_async()
            raise

    except Exception as e:
        print(f"\nOuter error occurred: {str(e)}")
        raise
    finally:
        await db.close_async()
        print("\n=== Transaction Demo Completed ===")


if __name__ == "__main__":
    asyncio.run(demonstrate_basic_transaction_async())
```

🔍 Key Migration Changes {#-key-migration-changes}

Looking at these examples, you can see several key changes made during the migration:

1. **Database Engine Change**:
   - Changed from `TargetDatabaseEngine.MSSQL` to `TargetDatabaseEngine.PSYCOPG3`
2. **Connection Parameters**:
   - Removed `DRIVER` parameter (not needed for PostgreSQL)
   - Changed port from 13001 (SQL Server) to 5432 (PostgreSQL)
   - Updated database name
3. **SQL Syntax Modifications**:
   - Added double quotes around table and column names: `"Production"."Product"` instead of `Production.Product`

- Changed parameter placeholders from `?` to `%s`
- Updated date functions: `SYSDATETIME()` → `CURRENT_TIMESTAMP`
- Changed interval addition: `DATEADD(day, 7, SYSDATETIME())` → `CURRENT_TIMESTAMP + INTERVAL '7 days'`
- Changed identity retrieval: `IDENT_CURRENT('Production.WorkOrder')` → `currval(pg_get_serial_sequence('"Production"."WorkOrder"', '"WorkOrderID"'))`

4. **Async Implementation**:

   - Added `use_async=True` parameter to constructor
   - Added await to async method calls
   - Used async context managers: `async with`
   - Used transaction context management for async transactions
   - Changed method names to async versions

By following this pattern, you can migrate any SQL Server database code to work with PostgreSQL and psycopg3, leveraging the full power of asynchronous operations when needed.

📈 Performance Considerations {#-performance-considerations}

When using psycopg3 with asynchronous operations, keep these performance considerations in mind:

1. **Connection Pooling**:

   - For production environments, use connection pooling to avoid the overhead of establishing new connections
   - Configure pool sizes based on your application's concurrency requirements

2. **Batch Operations**:

   - Use batch operations for inserting or updating multiple rows
   - Leverage `execute_batch` or `execute_values` for improved performance

3. **Prepared Statements**:

   - Use prepared statements for frequently executed queries
   - With psycopg3, prepared statements are handled automatically

4. **Async Execution**:

   - Use async execution for I/O-bound operations
   - Consider using asyncio's `gather` for concurrent operations

5. **Transaction Management**:

   - Use transactions to group related operations
   - Leverage context managers for cleaner transaction handling

6. **Result Processing**:

   - Process results in chunks for large result sets
   - Use server-side cursors for very large result sets

By following these best practices, you can get the most out of psycopg3's performance capabilities, especially in asynchronous scenarios.

Security Considerations {#-security-considerations}

When migrating to psycopg3, ensure you maintain proper security practices:

1. **Parameter Binding**:
    - Always use parameter binding (`%s`) instead of string concatenation to prevent SQL injection
    - Ensure all user input is properly parameterized
2. **Connection Credentials**:
    - Use environment variables or secure vaults (not hardcoded values)
    - Follow the principle of least privilege for database users
3. **SSL/TLS**:
    - Enable SSL connections to PostgreSQL for data in transit protection
    - Configure `sslmode` in connection parameters
4. **Connection Timeouts**:
    - Set appropriate connection and statement timeouts
    - Handle connection failures gracefully
5. **Error Handling**:
    - Implement proper error handling to avoid exposing sensitive information
    - Log errors securely (without sensitive data)
6. **Data Access Controls**:
    - Implement row-level security if needed
    - Use PostgreSQL roles and permissions effectively

By addressing these security concerns during the migration, you can ensure your application remains secure while benefiting from the improved capabilities of psycopg3.

Final Thoughts {#-final-thoughts}

Migrating from psycopg2 to psycopg3 with asynchronous capabilities represents a significant upgrade to your database interaction layer. This migration brings several advantages:

- **Improved Performance**: Asynchronous operations allow your application to handle more concurrent database operations without blocking.
- **Modern Python Features**: Integration with Python's async/await syntax results in cleaner, more maintainable code.
- **Better Resource Utilization**: Connection pooling and efficient resource management improve overall application performance.

- **Future-Proofing**: Alignment with modern Python and PostgreSQL practices ensures long-term sustainability.

By following this guide, you've successfully migrated from a SQL Server-based application to a PostgreSQL-based application with advanced asynchronous capabilities, setting a solid foundation for building high-performance database applications.

Remember to thoroughly test your migrated code in a staging environment before deploying to production, and consider implementing a gradual migration strategy for large applications to minimize risk.

Back to Table of Contents